

Automated Classification of Intermittent Pulsars Utilising Convolutional Neural Networks

Student Number: 100403391

School of Engineering, Mathematics, and Physics
University of East Anglia

Abstract

Intermittent pulsars present a significant challenge for radio astronomers. Their emissions are unpredictable and frequently blend into background noise, causing traditional amplitude-based filters to fail. This project developed a Convolutional Neural Network to classify radio signals based purely on their geometric shape rather than raw brightness. The network was trained on stable emissions from PSR J1910+0517 and tested on the variable emissions from PSR J1929+1357. Inverse-frequency class weights were applied to address the imbalance between noise and genuine signals. Evaluated on the unseen archive, the algorithm achieved a 94 per cent recall rate. Merging both pulsars into a unified training dataset yielded an overall accuracy of 97 per cent. This balance of sensitivity and precision indicates that deep learning offers an effective automated classification tool for future large-scale sky surveys.

Keywords: Pulsars, Intermittent Pulsars, Deep Learning, Convolutional Neural Networks, Radio Astronomy, Signal Classification.

Contents

1	Introduction	7
1.1	Astrophysical Context and Neutron Star Formation	7
1.2	Rotational Dynamics and Mathematical Parameters	7
1.3	Intermittent Emission and Observational Challenges	8
1.4	The Classification Bottleneck and Statistical Methodologies	8
1.5	Project Rationale and Primary Objectives	9
1.6	Principal Findings and Document Roadmap	9
2	Methodology and Data Processing	10
2.1	Research Design	10
2.2	Data Acquisition and Sources	10
2.3	Dimensionality Reduction	10
2.4	Phase Bin Standardisation	11
2.5	Amplitude Normalisation Mathematics	12
2.6	Ground Truth Labelling and Dataset Integration	12
2.7	Methodological Limitations	13
3	Convolutional Neural Network Architecture	14
3.1	Theoretical Foundations of One-Dimensional Networks	14
3.2	The Convolution Operation and Backpropagation	14
3.3	Non-Linear Activation with Rectified Linear Units	14
3.4	Spatial Down-Sampling with Max-Pooling	15
3.5	Regularisation with Dropout	15
3.6	Dense Classification and Output Layer	16
3.7	Complete Network Topology	16
4	Class Weighting and Training Dynamics	18
4.1	Experimental Paradigms and Data Partitioning	18
4.2	Dataset Imbalance and Class Weighting	18
4.3	Binary Cross-Entropy Loss Function	19
4.4	The Adam Optimiser and Learning Rate	19
4.5	Epochs and Batch Size Configuration	20
4.6	Evaluation Metrics	20
5	Results and Generalisation Testing	21
5.1	Evaluation Metrics	21
5.2	Scenario 1: Forward Generalisation Results	21
5.3	Scenario 2: Reverse Generalisation Results	23
5.4	Scenario 3: Unified Generalisation Results	25
6	Discussion	28
6.1	Interpretation of Forward Generalisation	28
6.2	Interpretation of Reverse Generalisation	28
6.3	The Efficacy of the Unified Approach	28

- 6.4 Comparison with Existing Literature 29
- 6.5 Cross-Disciplinary Parallels in Time-Series Analysis 29
- 6.6 Methodological Limitations 29
- 7 Conclusions and Future Work 31**
 - 7.1 Conclusions 31
 - 7.2 Future Work 31
- A Project Source Code 35**
 - A.1 Model Architecture and Training Script 35

List of Figures

1	A visual comparison of normalised one-dimensional phase profiles from PSR J1910+0517 and PSR J1929+1357, showing the topological differences between genuine pulses and background noise.	13
2	A computational summary of the one-dimensional Convolutional Neural Network architecture generated via the Keras library, detailing the sequential flow of data, output shapes, and total trainable parameters.	17
3	Training and validation metrics for Scenario 1. The left-hand plot displays the Binary Cross-Entropy loss, and the right-hand plot displays the recall sensitivity across the 50 epochs.	22
4	Confusion matrix for Scenario 1, detailing the distribution of true and false classifications on the unseen PSR J1929+1357 archive.	23
5	Training and validation metrics for Scenario 2. The left-hand plot displays the Binary Cross-Entropy loss, and the right-hand plot displays the recall sensitivity across the 50 epochs.	24
6	Confusion matrix for Scenario 2, detailing the distribution of true and false classifications on the unseen PSR J1910+0517 archive.	25
7	Training and validation metrics for the Unified Scenario. The left-hand plot displays the Binary Cross-Entropy loss, and the right-hand plot displays the recall sensitivity across the 50 epochs.	26
8	Confusion matrix generated from the unified holdout dataset, detailing the exact distribution of true positives, false positives, true negatives, and false negatives across the 650 unseen observations.	27

List of Tables

1	Computational logs at 5-epoch intervals for Scenario 1.	21
2	Classification report for the 2955 samples in the Scenario 1 testing dataset.	22
3	Computational logs at 5-epoch intervals for Scenario 2.	24
4	Classification report for the 295 samples in the Scenario 2 testing dataset.	24
5	Extracted computational logs at 5-epoch intervals for the Unified Scenario.	26
6	Classification report for the 650 samples in the Unified testing dataset, detailing precision, recall, and F1-scores for both the noise and pulse classes.	26

1 Introduction

1.1 Astrophysical Context and Neutron Star Formation

The existence of highly dense neutron stars was first theorised by Baade and Zwicky shortly after the discovery of the neutron [1]. Following the observation of a pulsating radio source by Jocelyn Bell in 1967 and its subsequent publication, pulsars were identified as rapidly rotating, highly magnetised neutron stars [2]. These extreme stellar remnants form during the core-collapse supernovae of massive progenitor stars. As the stellar core is compressed by gravity to nuclear densities, conservation of angular momentum produces objects that can complete hundreds of rotations per second [3].

The periodic emission from these objects is commonly described by the lighthouse model. Rapid rotation combined with strong magnetic fields accelerates charged particles along the magnetic field lines [4]. This generates focused beams of electromagnetic radiation from the magnetic poles. When the rotation axis and magnetic axis are misaligned, the beams sweep across the sky. Radio telescopes detect a pulse only when one of these beams crosses the line of sight [5].

The pulsar population shows considerable diversity and is broadly grouped into three main classes according to the source of energy. Rotation-powered pulsars, the most common type, radiate through the gradual loss of rotational kinetic energy. Accretion-powered pulsars, usually seen as X-ray sources in binary systems, draw energy from material transferred from a companion star. Magnetars are powered by the decay of strong magnetic fields, producing intense outbursts of X-ray and gamma-ray radiation [6].

Further distinctions arise from evolutionary history and environment [7]. Isolated pulsars slow down gradually over time. Millisecond pulsars (MSPs) are recycled through mass transfer in binary systems, reaching periods of 1–10 milliseconds and exhibiting high rotational stability. Binary systems range from neutron star–white dwarf pairs in circular orbits to highly eccentric double neutron star systems such as the Hulse-Taylor pulsar [8]. Other notable classes include high-mass binaries with Be stars, spider pulsars that ablate their companions, planet-hosting pulsars, and double pulsars. Rotating Radio Transients (RRATs) emit sporadic bursts rather than steady periodic signals, posing additional detection challenges [9].

1.2 Rotational Dynamics and Mathematical Parameters

The collapse of a massive stellar core into a rapidly spinning neutron star provides a demonstration of conservation laws. Once nuclear fuel is exhausted, radiation pressure can no longer support the star, and the core collapses under gravity. The resulting neutron star is supported against further collapse by neutron degeneracy pressure [10]. Hydrostatic equilibrium in this relativistic regime is described by the Tolman–Oppenheimer–Volkoff (TOV) equation:

$$\frac{dP}{dr} = -\frac{G}{r^2} \left(\rho + \frac{P}{c^2} \right) \left(m + \frac{4\pi r^3 P}{c^2} \right) \left(1 - \frac{2Gm}{rc^2} \right)^{-1}, \quad (1)$$

where P is pressure, ρ is density, m is enclosed mass, and r is radius [11]. This balance compresses roughly 1.4 solar masses into a sphere of about 10 km radius.

Conservation of angular momentum ($L = I\omega$) during the collapse increases the rotation rate as the moment of inertia decreases [12]. The newborn neutron star can therefore rotate hundreds of times per second.

The evolutionary path of the remnant is tracked through its rotational parameters. Observatories measure the period P and spin-down rate $\dot{P}$. Energy loss through magnetic dipole radiation causes gradual deceleration [13]. The characteristic age is estimated as

$$\tau = \frac{P}{2\dot{P}}, \quad (2)$$

assuming magnetic dipole braking [14]. The surface magnetic field strength is approximated by

$$B \approx 3.2 \times 10^{19} \sqrt{P\dot{P}} \text{ Gauss}. \quad (3)$$

The $P-\dot{P}$ diagram serves as the main diagnostic tool in pulsar astronomy, separating ordinary pulsars from recycled millisecond pulsars and tracing their evolution toward the death line, beyond which rotational energy is insufficient to sustain observable emission.

1.3 Intermittent Emission and Observational Challenges

Although most catalogued pulsars show stable periodicity, a subset known as intermittent pulsars displays extreme variability. These objects exhibit nulling, in which radio emission ceases or is suppressed for intervals ranging from minutes to years. The 2017 PALFA survey discovery of two long-term intermittent sources, PSR J1910+0517 and PSR J1929+1357, marked a significant advance [15]. Both sources alternate between active phases lasting several months and quiescent phases of similar duration.

Researchers distinguish active (ON) and inactive (OFF) states using statistical methods applied to extended observations. Flux density histograms are fitted with a zero-centred Gaussian for the noise floor and a second distribution for active emission [16]. From these fits, the nulling fraction (proportion of missing pulses) and duty cycle (fraction of time in the active state) are derived.

1.4 The Classification Bottleneck and Statistical Methodologies

The unpredictable nulling behaviour of intermittent pulsars creates a major processing challenge for large-scale surveys. Traditional matched-filter pipelines depend on absolute signal-to-noise ratio thresholds and therefore frequently discard faint intermittent pulses obscured by noise or scintillation [17].

To reduce reliance on manual inspection, astronomers apply statistical clustering techniques using libraries such as `astroML` and `scikit-learn`. Gaussian Mixture Models and Extreme Deconvolution separate candidates in the $P-\dot{P}$ diagram, with the Bayesian Information Criterion guiding the number of clusters [18]. Gaussian Process regression models profile variations and spin-down changes, while two-dimensional Kolmogorov–Smirnov tests and Monte Carlo simulations of the Rotating Vector Model provide additional validation [19].

Despite their sophistication, these statistical methods remain sensitive to parameter choices and high-variance noise. They often assume clear boundaries between signal and noise and can misclassify scintillation spikes or terrestrial interference in raw telescope data.

1.5 Project Rationale and Primary Objectives

Accurate detection of pulsars, especially rare or intermittent types, is essential in modern astrophysics. Neutron stars offer unique laboratories for testing fundamental physics, constraining the equation of state of ultra-dense matter, enabling gravitational-wave detection through Pulsar Timing Arrays, and providing stringent tests of General Relativity in binary systems. Missing intermittent sources due to inadequate filtering therefore represents a significant loss of scientific data.

To address the shortcomings of amplitude-based thresholding and rigid statistical clustering, this project developed an automated classification framework based on supervised deep learning. Candidate signals are evaluated solely on the basis of geometric pulse morphology.

The specific aims were:

- Develop a machine learning pipeline for the automated binary classification of active (ON) and inactive (OFF) astronomical radio signals.
- Transform multi-dimensional telescope arrays into a standardised one-dimensional format using min-max normalisation to remove absolute flux density variations.
- Train a Convolutional Neural Network capable of identifying underlying pulse structures autonomously, without manual parameter tuning.
- Implement class-weighting to mitigate the severe dataset imbalance between abundant background noise and rare genuine pulses.
- Evaluate the generalisation capabilities of the model on unseen, highly variable pulsar archives under isolated and unified training scenarios.

1.6 Principal Findings and Document Roadmap

The research demonstrates that deep learning offers a viable approach for future large-scale sky surveys. The pipeline shows that a neural network can extract structural pulse features from a steady source and successfully generalise to a highly variable target. On the unified dataset the model achieved 97 per cent overall accuracy, indicating that convolutional architectures can balance sensitivity and precision effectively in high-noise environments.

The remainder of the dissertation is structured as follows. Section 2 details the data acquisition, dimensionality reduction, and standardisation methodologies. Section 3 outlines the architecture of the one-dimensional Convolutional Neural Network. Section 4 presents the application of class weights and the network training dynamics. Section 5 reports the experimental generalisation results and quantitative evaluation metrics. Section 6 provides a discussion of these findings alongside cross-disciplinary comparisons. Finally, Section 7 summarises the study and proposes directions for future research.

2 Methodology and Data Processing

2.1 Research Design

The main aim of this research was to develop an automated classification pipeline capable of detecting intermittent pulsars without relying on absolute signal brightness. A quantitative computational approach was adopted. Traditional radio astronomy pipelines evaluate candidate signals using amplitude thresholds based on the signal-to-noise ratio. As noted in the introduction, these methods frequently fail when applied to faint intermittent sources [17].

To address this limitation, the research design avoided amplitude thresholding altogether. A supervised machine learning approach was adopted instead, with classification performed solely on the basis of geometric topology. Convolutional architectures were chosen over traditional machine learning algorithms, such as random forests or decision trees, because these standard models struggle with raw spatial data and require extensive manual feature engineering. By extracting and standardising the one-dimensional shape of each pulse, a Convolutional Neural Network could learn to recognise the characteristic morphology of genuine astronomical signals. The local receptive fields and translation invariance of this network make it well suited to identifying pulse-like features amid background noise and minor phase shifts.

2.2 Data Acquisition and Sources

The observational data used in this study came from records of two intermittent pulsars: PSR J1910+0517 and PSR J1929+1357. Although these sources were originally discovered during the PALFA survey at the decommissioned Arecibo Observatory [15], the observations processed here were recorded during follow-up campaigns using the Lovell Telescope at Jodrell Bank Observatory. The two pulsars were chosen to test generalisation under contrasting conditions. PSR J1910+0517 presents a wide, faint, and relatively steady pulse profile, providing a stable baseline for training. PSR J1929+1357, by contrast, has a narrow, sharp profile that is strongly affected by interstellar scintillation.

The raw files were accessed from a secure Linux server in the university Engineering, Mathematics and Physics department. Exactly 3250 individual observation files were processed across both astronomical sources. The data were stored in standard pulsar archive format, which includes multiple dimensions such as integration time, frequency channels, and polarisation states.

2.3 Dimensionality Reduction

Raw pulsar observation files contain multi-dimensional arrays that record signal intensity across frequency channels, polarisation states, and time sub-integrations. The files from Jodrell Bank Observatory uniformly consisted of 60-second sub-integrations. The total observation lengths varied depending on the intrinsic signal strength of the specific target. Because PSR J1929+1357 emits a stronger signal, shorter observations ranging between 4 and 12 minutes were sufficient to return a clear pulse, with a standard duration of 6 minutes. Conversely, longer observation windows ranging from 15 to 30 minutes were required for PSR J1910+0517 to compensate for its weaker signal strength. Prior to computational processing, both automated and manual radio frequency interference excision were applied to the archives by previous researchers to remove corrupted data.

A dimensionality reduction protocol was applied to prepare the data for the one-dimensional convolutional neural network. A raw pulsar observation can be represented as a four-dimensional tensor $I(\nu, p, t, \phi)$, where ν denotes frequency channels, p denotes polarisation parameters, t denotes time sub-integrations, and ϕ denotes rotational phase bins. This data structure is heavily utilised in modern pulsar processing pipelines [20]. To focus the model exclusively on pulse shape detection, this tensor was collapsed into a one-dimensional array.

Reduction was performed using the PyPulse library in automated Python scripts. Three standard scrunching operations were carried out for each archive:

- Summed all Stokes parameters to yield total intensity, effectively removing the polarisation dimension p .
- Integrated all frequency channels into a single broadband profile by taking the sum over ν , removing frequency-dependent variations.
- Averaged all temporal sub-integrations by calculating the mean across t , producing one representative phase profile.

Through these integration operations, the original four-dimensional tensor was collapsed into a single one-dimensional intensity array $X(\phi)$ plotted as a function of the rotational phase.

2.4 Phase Bin Standardisation

Convolutional Neural Networks require inputs of fixed length. Raw phase profiles varied in bin resolution depending on the original telescope settings. To ensure uniformity, all profiles were resampled to exactly 512 phase bins. This resolution was selected as a power of two that balances computational memory efficiency with sufficient detail to capture critical pulse features.

Resampling was performed using SciPy signal processing routines via Fourier-based interpolation, an approach utilised in high-precision astrophysical time-series analysis [21]. To execute this, the original discrete signal $x[n]$ of length N was first transformed into the frequency domain using the Discrete Fourier Transform to generate complex coefficients $X[k]$:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-i2\pi kn/N} \quad (4)$$

To resample the array to the uniform target length of $M = 512$, a new frequency domain array $Y[k]$ of length M was constructed. If the original profile contained fewer bins than the target ($M > N$), the complex array $Y[k]$ was symmetrically zero-padded in the high-frequency centre. If the original profile contained more bins than the target ($M < N$), the highest frequency components were truncated. The Inverse Discrete Fourier Transform was then applied to map the modified data back into the time domain, yielding the standardised resampled array $y[m]$:

$$y[m] = \frac{1}{M} \sum_{k=0}^{M-1} Y[k] e^{i2\pi km/M} \quad (5)$$

This Fourier methodology interpolates the data points and preserves the overall geometric shape of the pulse, although it can occasionally introduce minor ringing artefacts near extremely sharp

transitions. Following this standardisation, every observation possessed a fixed input shape of 512×1 .

2.5 Amplitude Normalisation Mathematics

A central departure from conventional detection methods was the total removal of absolute flux density values. Because intermittent pulses are faint and affected by interstellar scintillation, their peak physical amplitudes vary widely. Retaining these raw intensities would have encouraged the neural network to learn numerical amplitude thresholds rather than geometric shapes.

The complete removal of this physical flux density was executed through min-max normalisation. In radio astronomy, raw signal amplitude is physically measured in Janskys (Jy). For a phase profile array X measured in Janskys, the dimensionless normalised array is given by

$$X_{\text{norm}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}} \quad (6)$$

where $X_{\min}$ and $X_{\max}$ are the minimum and maximum flux density values in the profile. This calculation removes the absolute density scale and maps all proportional values to a dimensionless range between 0 and 1.

Arrays consisting entirely of background noise where $X_{\max}$ equals $X_{\min}$ were set to absolute zero to prevent division by zero errors. This normalisation destroyed all absolute brightness information, presenting the network with pure topological structures. This exact normalisation strategy has been proven successful in prior machine learning studies, demonstrating that Convolutional Neural Networks achieve superior generalisation when forced to focus on structural arrays bounded between 0 and 1 [22].

2.6 Ground Truth Labelling and Dataset Integration

Supervised learning requires each example to be paired with an accurate ground truth label. The model improves by comparing its predictions to these labels and minimising the resulting error.

Ground truth labels were provided by Tibby Finn Leeming, an astrophysics doctoral student at the University of East Anglia, prior to this project. The labels were stored in text files linking each observation identifier to a binary value, where 1 indicated a genuine pulse and 0 indicated background noise only.

A custom Python function matched the processed one-dimensional arrays with their labels. After reduction, resampling, and normalisation, each label was appended to a master list. This yielded two distinct datasets: For PSR J1910+0517, exactly 295 observations were compiled containing 84 active pulses and 211 noise files. For PSR J1929+1357, exactly 2955 observations were compiled containing 174 active pulses and 2781 noise files. The arrays and labels were converted into NumPy tensors and saved to disk for use in training and testing.

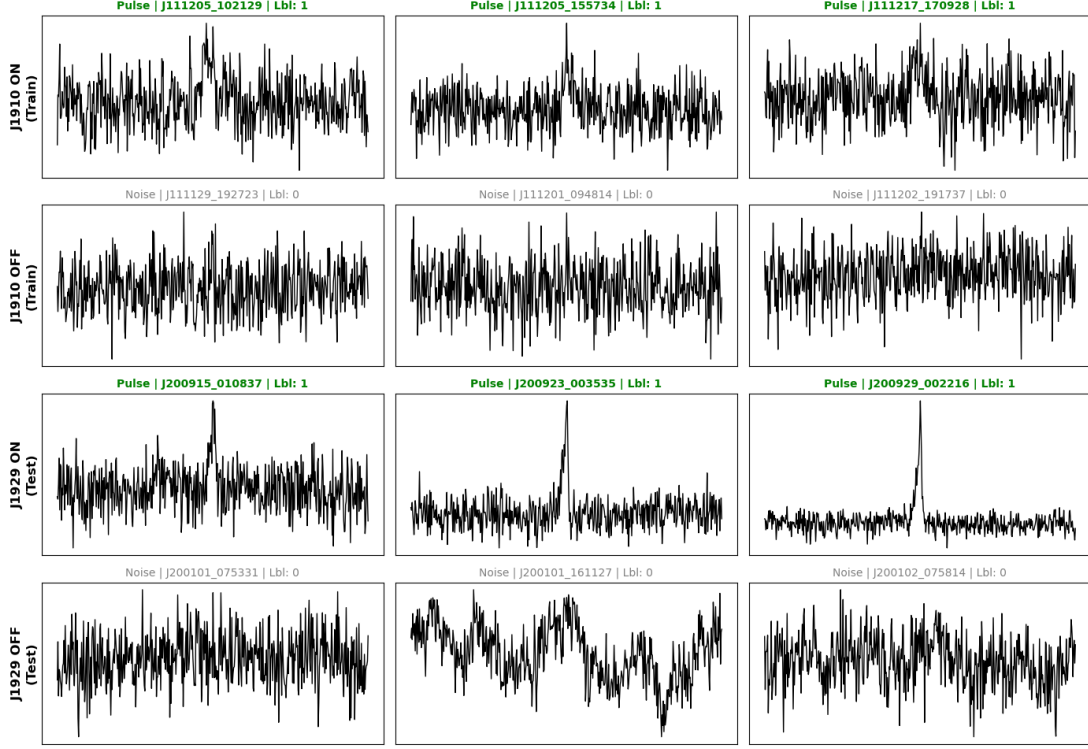


Figure 1: A visual comparison of normalised one-dimensional phase profiles from PSR J1910+0517 and PSR J1929+1357, showing the topological differences between genuine pulses and background noise.

2.7 Methodological Limitations

The data processing pipeline successfully isolates pulse topology, but it carries certain limitations. The most important drawback is the loss of frequency-dependent scintillation information due to frequency scrunching. Any differences in pulse behaviour across radio bands are permanently removed.

In standard astrophysics, frequency-resolved data help distinguish broadband pulsar emission from narrow-band terrestrial interference, as terrestrial signals rarely span the entire observation bandwidth simultaneously [23]. By reducing the data to one dimension, the network no longer has access to this diagnostic. Retaining frequency channels would have required a more complex two-dimensional architecture. Given the limited number of genuine intermittent pulses available for training, such a model would have been susceptible to severe overfitting. The one-dimensional approach was therefore the most practical and robust choice for meeting the primary research objectives.

3 Convolutional Neural Network Architecture

3.1 Theoretical Foundations of One-Dimensional Networks

The data processing pipeline produced standardised one-dimensional arrays of shape 512 by 1 that captured the geometric topology of each pulse. To classify these arrays effectively, a neural network architecture capable of preserving spatial relationships was required. Traditional fully connected networks flatten the input immediately, which discards the local relationships between adjacent phase bins. For pulsar data, these relationships, the rising slopes, peaks, and falling edges, are essential for distinguishing genuine pulses from noise.

A one-dimensional Convolutional Neural Network was therefore implemented. Unlike fully connected layers, convolutional layers maintain the sequential order of the input and apply shared filters that scan across the phase profile. This design enables the network to detect local geometric features regardless of their exact position. The resulting translation invariance is useful given timing noise and residual dispersion effects in pulsar observations.

3.2 The Convolution Operation and Backpropagation

The core operation in this architecture is the convolutional layer. A small kernel of learnable weights is slid sequentially across the input array [24]. At each spatial position, the dot product between the kernel weights and the corresponding data segment is computed, producing a feature map. For a one-dimensional input array x and a kernel w of length k , the output feature map y at position i is given by

$$y[i] = \sum_{j=0}^{k-1} x[i+j] \cdot w[j]. \quad (7)$$

During the training phase, the kernel weights are updated through backpropagation. This process calculates how much each individual weight contributed to the overall prediction error by applying the calculus chain rule [25]. For a defined loss function L , the gradient of the loss with respect to a specific weight w is calculated as the product of the partial derivatives across the network layers:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial w}. \quad (8)$$

Once the gradient is obtained, the weight is updated by subtracting a fraction of the gradient scaled by the learning rate. Through this iterative process, early convolutional layers typically learn simple features such as edges or sharp gradients, while deeper layers combine these to recognise more complex, asymmetrical pulse topologies [26].

3.3 Non-Linear Activation with Rectified Linear Units

Convolution is a linear operation [27]. To enable the network to model complex non-linear relationships, a non-linear activation function is applied immediately after each convolutional layer. The Rectified Linear Unit (ReLU) was used exclusively throughout the hidden layers. ReLU is defined mathematically as

$$f(x) = \max(0, x). \quad (9)$$

This function forces all negative values to zero while passing positive values unchanged. ReLU was selected over alternatives such as the hyperbolic tangent because its simple gradient of 1 for positive inputs helps prevent the vanishing gradient problem [28]. This keeps weight updates stable as the backpropagation chain rule travels deeper into the network.

3.4 Spatial Down-Sampling with Max-Pooling

Each convolutional layer generates a large number of feature maps. To reduce computational cost and retain only the most salient geometric features, one-dimensional max-pooling layers were inserted after the convolutional blocks.

A max-pooling layer slides a fixed window of size p across the feature map y using a specified step size known as a stride s . For each window, the pooling operation extracts the maximum numerical value and discards the rest. Mathematically, the pooled output z at index i is defined as:

$$z[i] = \max_{0 \leq j < p} (y[i \cdot s + j]). \quad (10)$$

In this architecture, both the window size and stride were set to 2. This configuration was chosen because it offers an efficient spatial reduction while retaining sufficient detail in the phase profile. A larger window size, such as 4, would compress the one-dimensional array more aggressively and could merge narrow genuine pulsar spikes with nearby background noise. Setting the stride equal to the window size ensures that the pooling operation divides the data without overlap or gaps.

This step halves the spatial dimensions at each application, which lowers the computational demand and helps limit overfitting in the later dense layers. By keeping only the strongest feature activations within each window, the network gains robustness to minor phase shifts and topological distortions introduced by interstellar scintillation.

3.5 Regularisation with Dropout

Astronomical datasets are typically imbalanced, which increases the risk of overfitting. Overfitting occurs when the network memorises the specific noise patterns of the training data rather than learning universal, generalisable pulse characteristics [29].

To mitigate this risk, inverted dropout was applied within the fully connected dense layers. During each training epoch, a random subset of neurons is temporarily disabled. This process follows a Bernoulli probability distribution, where each neuron has a probability p of being retained. To ensure the overall expected value of the network activations remains constant during both training and testing, the retained activations x are scaled by the retention probability. For a Bernoulli mask r , the regularised output y is computed as:

$$y = \frac{r \cdot x}{p}. \quad (11)$$

In this architecture, a dropout rate of 50 percent was used. This operation prevents neurons from co-adapting and encourages the network to distribute learned topological features across multiple pathways, thereby improving generalisation to unseen pulsar archives.

3.6 Dense Classification and Output Layer

After the final convolutional and pooling layers, the compressed feature maps are flattened into a single vector. This vector is passed through one or more fully connected dense layers that integrate the extracted spatial features to form the final classification decision.

Because the task is binary, the final output layer consists of a single neuron with a sigmoid activation function:

$$\sigma(x) = \frac{1}{1 + \exp(-x)}. \quad (12)$$

The sigmoid function scales the network output into a probability score between 0 and 1. A threshold of 0.5 was used: values above this threshold classify the input as an active pulse, while values below are classified as background noise.

3.7 Complete Network Topology

The final architecture was developed through iterative testing and is summarised below for reproducibility:

- Receives the standardised input phase profile of shape 512 by 1.
- Executes a one-dimensional convolutional layer with 16 filters, a kernel size of 16, and ReLU activation.
- Applies a one-dimensional max-pooling layer with a pool size of 4.
- Executes a second convolutional layer with 32 filters, a kernel size of 8, and ReLU activation.
- Applies a second max-pooling layer with a pool size of 4.
- Flattens the feature maps into a single vector.
- Passes the vector through a dense layer with 64 neurons and ReLU activation.
- Applies a dropout layer with a rate of 50 percent.
- Passes the data to the final dense output neuron with sigmoid activation to generate the binary probability score.

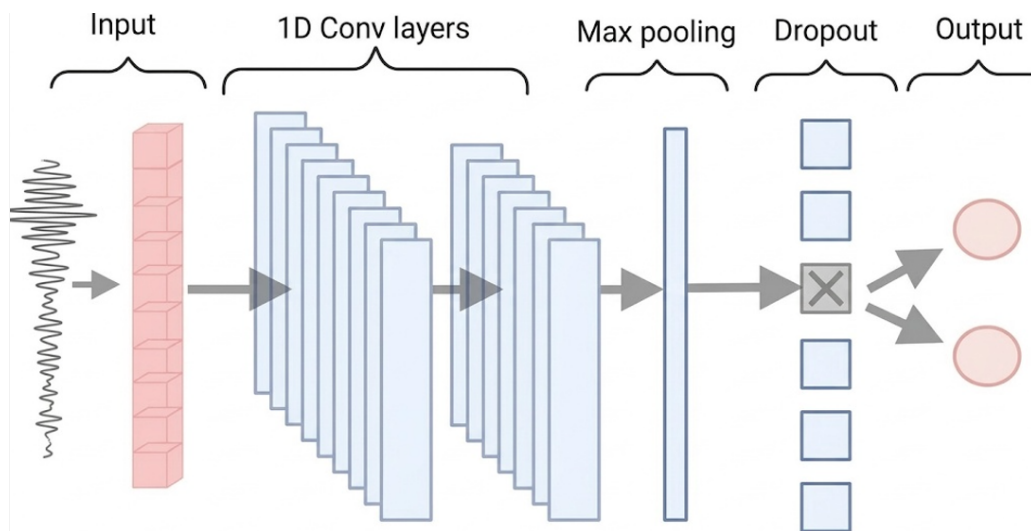


Figure 2: A computational summary of the one-dimensional Convolutional Neural Network architecture generated via the Keras library, detailing the sequential flow of data, output shapes, and total trainable parameters.

4 Class Weighting and Training Dynamics

4.1 Experimental Paradigms and Data Partitioning

Before training could begin, the datasets required careful partitioning to enable fair evaluation and to test the true generalisation capabilities of the neural network. To assess how well the algorithm could learn universal pulsar topologies rather than memorising a single source, three distinct experimental scenarios were established:

- **Scenario 1 (Forward Generalisation):** Trained the network on the stable dataset of PSR J1910+0517 and tested the model entirely on the unseen, highly variable dataset of PSR J1929+1357.
- **Scenario 2 (Reverse Generalisation):** Trained the network on the highly variable PSR - J1929+1357 dataset and tested the model entirely on the unseen stable PSR J1910+0517 dataset.
- **Scenario 3 (Unified Approach):** Merged both pulsar datasets into a single comprehensive archive. To evaluate overall performance, 20 percent of this unified archive was permanently extracted as an unseen holdout testing subset, leaving 80 percent available for the active training phase.

During the training process for all three scenarios, the active training data was subjected to an internal validation split. Exactly 80 percent of the allocated training arrays were used to update the network weights, while the remaining 20 percent were held back to monitor the validation loss at the end of each epoch.

To perform these dataset divisions, stratified sampling was implemented [30]. Stratified sampling ensures that the original prior probability distribution of the classes is preserved across all allocated data splits. For a given dataset containing a total of N observations, let N_c represent the total number of observations belonging to a specific class c . If a subset is required to contain a fraction f of the total data, the exact number of allocated samples for that class, n_c , is computed using the following equation:

$$n_c = \lfloor f \cdot N_c \rfloor \quad (13)$$

where $\lfloor \cdot \rfloor$ denotes rounding to the nearest whole integer. Within this research, the unified archive contained exactly 258 genuine pulses and 2992 background noise files. Consequently, active pulses constituted approximately 7.9 percent of the total data ($N_{pulse}/N \approx 0.079$). When generating the 20 percent testing subset ($f = 0.2$), stratified sampling calculated the allocation of exactly 52 genuine pulses and exactly 598 background noise files. This segregation prevents the rare astronomical events from being concentrated entirely within a single subset.

4.2 Dataset Imbalance and Class Weighting

Pulsar datasets are typically characterised by a strong imbalance, with noise examples far outnumbering genuine pulses [31]. In all three experimental scenarios, the inactive noise arrays outnumbered the active pulse arrays. Without intervention, a neural network tends to achieve high apparent accuracy simply by predicting the majority class for every input, thereby ignoring the rare positive examples.

To address this imbalance, class weighting was applied. Inverse class frequencies were calculated using the `compute_class_weight` algorithm from the scikit-learn software library. For a dataset containing a total of N observations divided into k unique classes, the weight W_c assigned to class c , which contains N_c observations, is defined by the following equation:

$$W_c = \frac{N}{k \times N_c} \quad (14)$$

Higher weights were assigned to the minority pulse class and lower weights to the majority noise class. These weights were passed to the loss function during training, increasing the penalty for misclassifying genuine pulses and encouraging the network to learn geometric features rather than defaulting to the majority class.

4.3 Binary Cross-Entropy Loss Function

The network requires a loss function to quantify the difference between its predictions and the ground truth labels. Because the task is binary, Binary Cross-Entropy was selected. For a true label y and predicted probability $\hat{y}$, the loss L across N samples is given by

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]. \quad (15)$$

This loss function penalises confident but incorrect predictions and works well with the sigmoid output, providing stable gradients during optimisation.

4.4 The Adam Optimiser and Learning Rate

The Adaptive Moment Estimation algorithm (Adam) is an advanced stochastic optimisation framework [32]. Adam computes individual adaptive learning rates for different parameters rather than using a single static rate. Within this architecture, Adam was used to iteratively update the internal weights of the convolutional and dense layers in order to minimise the Binary Cross-Entropy loss.

Adam achieves this adaptation by calculating exponential moving averages of the gradient and its square. At any given training step t , the algorithm computes the first moment m_t and the second moment v_t of the gradient g_t . These moments are updated using specific exponential decay rates β_1 and β_2 :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (16)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (17)$$

Bias-corrected estimates are calculated for both moments:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (18)$$

The network weight w_t is then updated according to the base learning rate η and a small smoothing term ϵ :

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \quad (19)$$

This behaviour helps the network navigate the complex loss surfaces associated with astronomical

data. For this pipeline, the default values for the exponential decay rates ($\beta_1 = 0.9$ and $\beta_2 = 0.999$) were retained. The base learning rate (η) was set to 0.001 following empirical testing. Higher values caused unstable oscillations in the loss, while lower values resulted in slow convergence. The selected rate of 0.001 provided a suitable balance between training speed and stability across all three experimental scenarios.

4.5 Epochs and Batch Size Configuration

Training was performed with a batch size of 16. This relatively small value provided a reliable gradient estimate while allowing the network to update its weights frequently enough to navigate the imbalanced data.

The training process was limited to 50 epochs across all experimental scenarios. Each epoch represents one complete pass through the training subset, this duration allowed the network to converge on the underlying pulse topologies without further reduction in validation loss. Extending training beyond 50 epochs showed no additional benefit and risked overfitting. The combination of batch size 16 and 50 epochs produced an efficient and stable training regime.

4.6 Evaluation Metrics

Performance evaluation relied upon a confusion matrix, alongside precision, recall, and F1-score metrics.

For true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN), the core metrics are defined by the following equations:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (20)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (21)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (22)$$

Balancing false positive and false negative rates requires an aggregate metric. The F1-score, representing the harmonic mean of precision and recall, is defined as [33]:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (23)$$

5 Results and Generalisation Testing

5.1 Evaluation Metrics

The algorithm was evaluated using a confusion matrix together with precision, recall, and F1-score.

Precision measures the proportion of predicted positive cases (pulses) that were actually correct. Recall (also called sensitivity) measures the proportion of actual positive cases that the model correctly identified. The F1-score is the mean of precision and recall, providing a single value that balances both metrics. Support indicates the number of true instances for each class in the test set.

Training progress was monitored with the following columns: Epoch (training iteration), Loss (Binary Cross-Entropy value), Accuracy (overall correct predictions), Recall (sensitivity on the pulse class), Val Loss / Val Accuracy / Val Recall (corresponding values on the validation set).

5.2 Scenario 1: Forward Generalisation Results

Forward generalisation testing involved training the network on stable observations from PSR J1910+0517 and evaluating it against variable emissions from PSR J1929+1357. Data partitioning yielded 236 training samples (169 noise profiles, 67 genuine pulses) and 59 validation samples (17 pulses).

Classification bias was mitigated by calculating class weights exclusively from the training subset. Applying Equation 14, the applied weights were:

$$W_{\text{noise}} = \frac{236}{2 \times 169} = 0.6982 \quad (24)$$

$$W_{\text{pulse}} = \frac{236}{2 \times 67} = 1.7612 \quad (25)$$

Training progression, tracking Binary Cross-Entropy loss alongside accuracy and recall across 50 epochs, is summarised in Table 1.

Table 1: Computational logs at 5-epoch intervals for Scenario 1.

Epoch	Loss	Accuracy	Recall	Val Loss	Val Accuracy	Val Recall
1	0.6997	0.5932	0.2239	0.6744	0.7119	0.0000
5	0.6434	0.5636	0.7612	0.5797	0.7119	0.0588
10	0.3983	0.8729	0.8060	0.4920	0.8305	0.7059
15	0.3276	0.9153	0.8060	0.4678	0.8305	0.7059
20	0.2528	0.9195	0.8507	0.4170	0.8305	0.6471
25	0.2354	0.9237	0.8955	0.4435	0.8305	0.7059
30	0.1859	0.9492	0.8955	0.4183	0.8475	0.7647
35	0.1583	0.9534	0.9254	0.4193	0.8644	0.7059
40	0.1366	0.9576	0.9104	0.4344	0.8475	0.7647
45	0.1011	0.9661	0.9701	0.4406	0.8644	0.7059
50	0.0839	0.9788	0.9701	0.4558	0.8644	0.7059

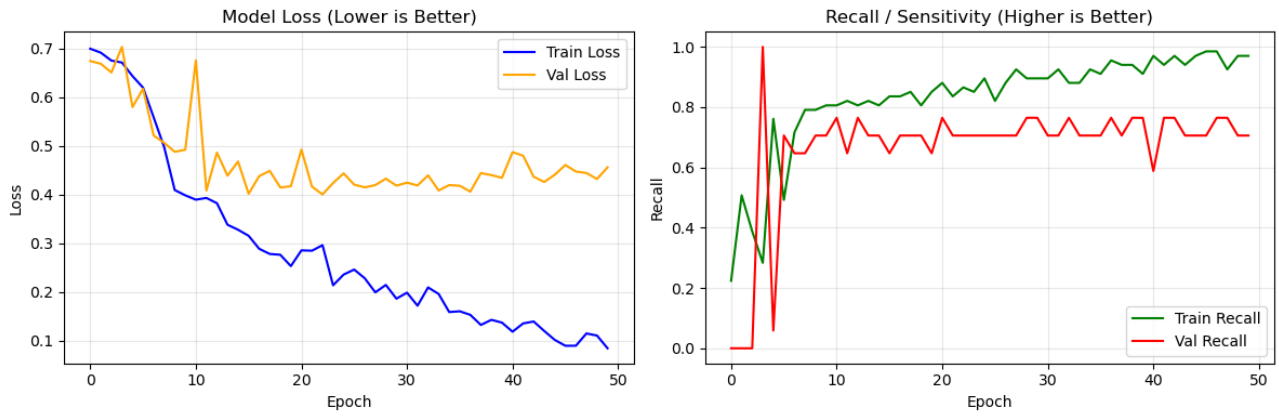


Figure 3: Training and validation metrics for Scenario 1. The left-hand plot displays the Binary Cross-Entropy loss, and the right-hand plot displays the recall sensitivity across the 50 epochs.

Subsequent evaluation applied the frozen model to the 2955 unseen testing arrays from PSR J1929+1357. The resulting performance metrics are detailed in Table 2.

Table 2: Classification report for the 2955 samples in the Scenario 1 testing dataset.

Class	Precision	Recall	F1-Score	Support
Noise (0)	1.00	0.95	0.97	2781
Pulse (1)	0.53	0.94	0.68	174
Accuracy		0.95		2955
Macro Avg	0.77	0.94	0.83	2955
Weighted Avg	0.97	0.95	0.95	2955

Prediction distributions are visualised in Figure 4 and broken down as follows:

- Detected 163 true pulses correctly (True Positives).
- Missed 11 real pulses (False Negatives).
- Flagged 142 background noise profiles incorrectly (False Positives).
- Ignored 2639 true negatives correctly (True Negatives).

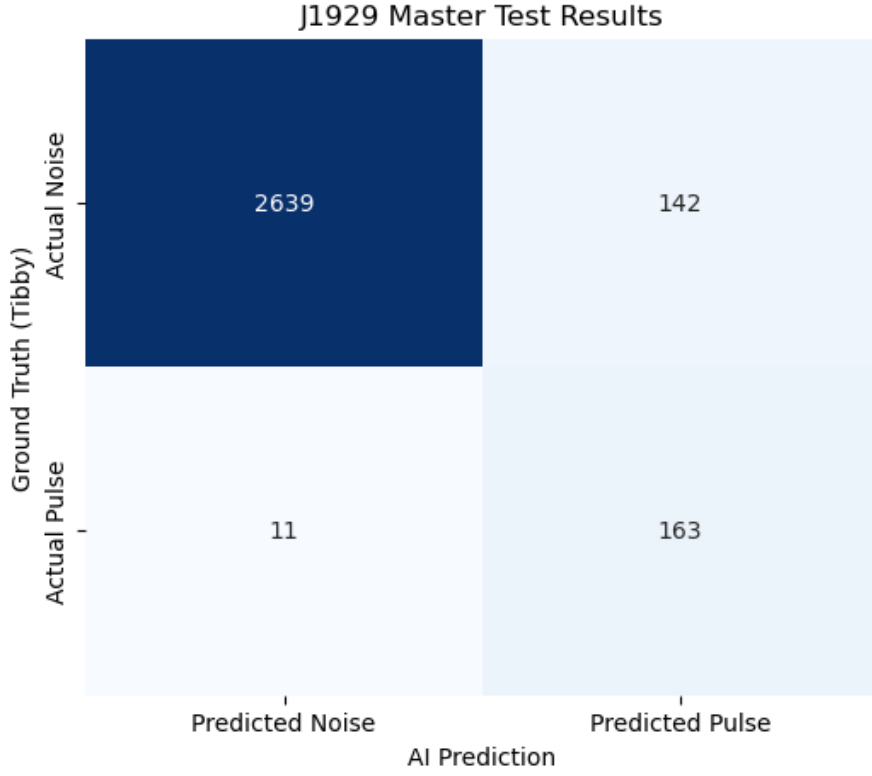


Figure 4: Confusion matrix for Scenario 1, detailing the distribution of true and false classifications on the unseen PSR J1929+1357 archive.

5.3 Scenario 2: Reverse Generalisation Results

Reverse generalisation inverted the experimental parameters. The 2955 observations from PSR J1929+1357 were partitioned into 2364 training samples (2225 noise profiles, 139 pulses) and 591 validation samples (35 pulses).

Class weights were derived from the training subset using Equation 14:

$$W_{\text{noise}} = \frac{2364}{2 \times 2225} = 0.5312 \quad (26)$$

$$W_{\text{pulse}} = \frac{2364}{2 \times 139} = 8.5036 \quad (27)$$

Epoch convergence logs are detailed in Table 3.

Table 3: Computational logs at 5-epoch intervals for Scenario 2.

Epoch	Loss	Accuracy	Recall	Val Loss	Val Accuracy	Val Recall
1	0.5410	0.9480	0.4388	0.2128	0.9729	0.9714
5	0.2486	0.9763	0.8921	0.2060	0.9645	0.9714
10	0.1877	0.9776	0.9065	0.1632	0.9831	0.9714
15	0.0996	0.9818	0.9568	0.0792	0.9831	0.9714
20	0.0423	0.9869	0.9928	0.0426	0.9915	0.9429
25	0.0373	0.9911	0.9856	0.0490	0.9882	0.9143
30	0.0504	0.9856	0.9712	0.0610	0.9848	0.9429
35	0.0094	0.9979	1.0000	0.0555	0.9882	0.9143
40	0.0203	0.9924	0.9856	0.1065	0.9645	0.9714
45	0.0045	0.9983	1.0000	0.0723	0.9848	0.8571
50	0.0046	0.9987	1.0000	0.0720	0.9865	0.8857

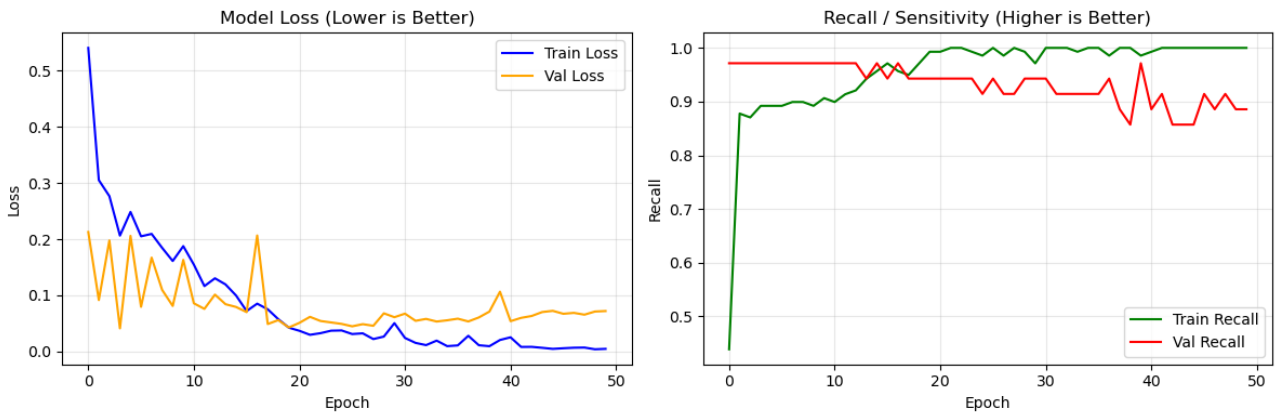


Figure 5: Training and validation metrics for Scenario 2. The left-hand plot displays the Binary Cross-Entropy loss, and the right-hand plot displays the recall sensitivity across the 50 epochs.

Application of the trained architecture to the 295 unseen testing arrays from PSR J1910+0517 yielded an overall accuracy of 76 percent. Full metrics are provided in Table 4.

Table 4: Classification report for the 295 samples in the Scenario 2 testing dataset.

Class	Precision	Recall	F1-Score	Support
Noise (0)	0.75	1.00	0.86	211
Pulse (1)	1.00	0.17	0.29	84
Accuracy		0.76		295
Macro Avg	0.88	0.58	0.57	295
Weighted Avg	0.82	0.76	0.69	295

Distribution of the secondary holdout classifications is mapped in Figure 6, comprising:

- Identified 14 true pulses correctly (True Positives).
- Missed 70 real pulses (False Negatives).

- Produced 0 incorrect noise classifications (False Positives).
- Filtered 211 true negatives correctly (True Negatives).

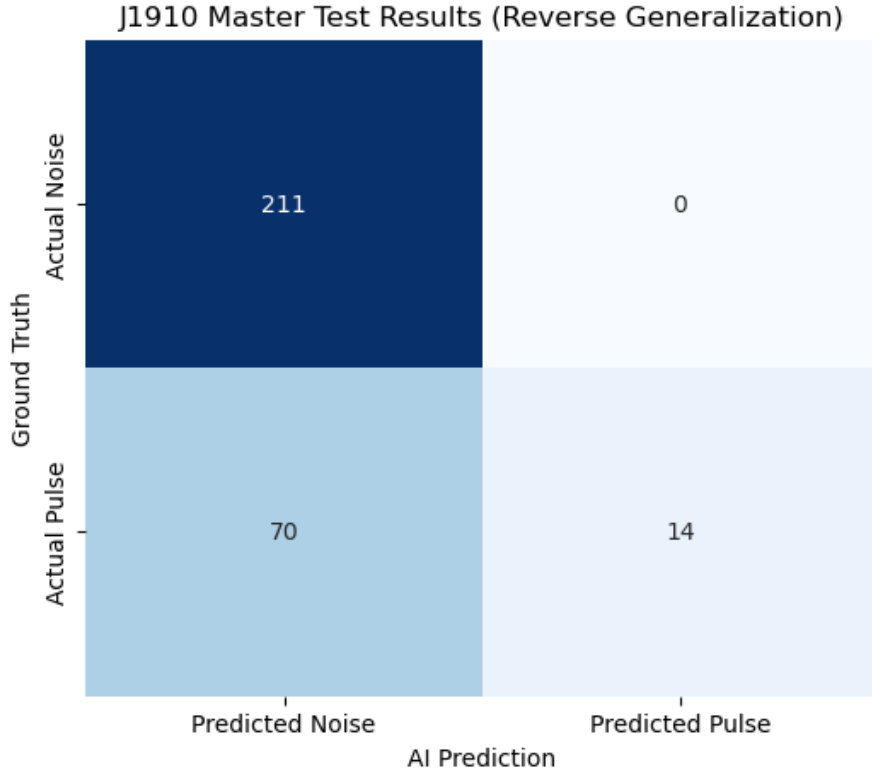


Figure 6: Confusion matrix for Scenario 2, detailing the distribution of true and false classifications on the unseen PSR J1910+0517 archive.

5.4 Scenario 3: Unified Generalisation Results

Evaluation of the unified approach merged all 3250 observations, dividing them into 2600 training samples (2394 noise profiles, 206 genuine pulses) and 650 testing samples (52 pulses).

Class weights derived from the training set were computed using Equation 14:

$$W_{\text{noise}} = \frac{2600}{2 \times 2394} = 0.5430 \quad (28)$$

$$W_{\text{pulse}} = \frac{2600}{2 \times 206} = 6.3107 \quad (29)$$

Convergence over the 50-epoch cycle is documented in Table 5.

Table 5: Extracted computational logs at 5-epoch intervals for the Unified Scenario.

Epoch	Loss	Accuracy	Recall	Val Loss	Val Accuracy	Val Recall
1	0.5817	0.7935	0.6068	0.4393	0.9415	0.8462
5	0.2705	0.9469	0.8544	0.4535	0.8677	0.8846
10	0.1947	0.9677	0.8883	0.1905	0.9585	0.8654
15	0.1707	0.9662	0.9029	0.1802	0.9477	0.8654
20	0.1072	0.9727	0.9272	0.1420	0.9600	0.8654
25	0.0959	0.9731	0.9417	0.1466	0.9631	0.8462
30	0.0609	0.9827	0.9660	0.1538	0.9631	0.8462
35	0.0604	0.9762	0.9709	0.1832	0.9569	0.8654
40	0.0395	0.9858	0.9806	0.1908	0.9554	0.8462
45	0.0271	0.9915	0.9903	0.1779	0.9769	0.8269
50	0.0259	0.9935	0.9854	0.1996	0.9738	0.8269

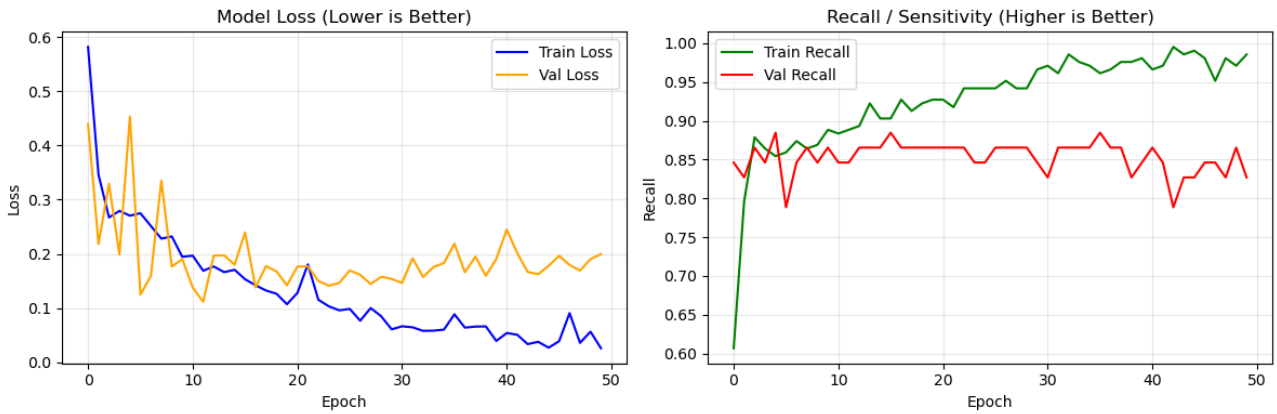


Figure 7: Training and validation metrics for the Unified Scenario. The left-hand plot displays the Binary Cross-Entropy loss, and the right-hand plot displays the recall sensitivity across the 50 epochs.

Final testing subjected the 650 holdout samples to the converged model. Table 6 outlines the classification report, demonstrating a 97 percent unified accuracy.

Table 6: Classification report for the 650 samples in the Unified testing dataset, detailing precision, recall, and F1-scores for both the noise and pulse classes.

Class	Precision	Recall	F1-Score	Support
Noise (0)	0.98	0.99	0.99	598
Pulse (1)	0.84	0.83	0.83	52
Accuracy		0.97		650
Macro Avg	0.91	0.91	0.91	650
Weighted Avg	0.97	0.97	0.97	650

Distribution of the holdout classifications is mapped in Figure 8, comprising:

- Classified 43 true pulses correctly (True Positives).
- Missed 9 real pulses (False Negatives).

- Categorised 8 background noise profiles incorrectly (False Positives).
- Rejected 590 true negatives correctly (True Negatives).

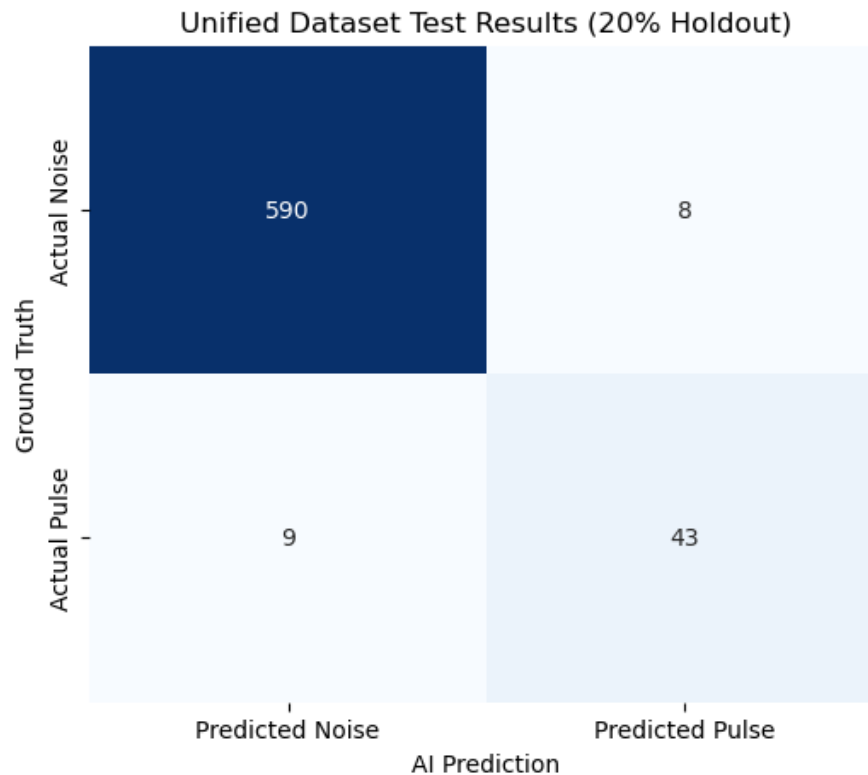


Figure 8: Confusion matrix generated from the unified holdout dataset, detailing the exact distribution of true positives, false positives, true negatives, and false negatives across the 650 unseen observations.

6 Discussion

6.1 Interpretation of Forward Generalisation

The project aimed to automate intermittent pulsar classification by focusing on pulse morphology rather than absolute signal amplitude. In Scenario 1 (forward generalisation), the network was trained exclusively on the broad, stable emissions of PSR J1910+0517 and tested on the unseen, scintillating emissions of PSR J1929+1357. It achieved a 94 per cent recall rate. This high sensitivity indicates that the convolutional layers captured the underlying Gaussian structure of an integrated pulse profile.

Precision, however, was lower. Because the training data lacked distorted profiles, the network became overly permissive, incorrectly flagging 142 noise anomalies as genuine pulses. This discrepancy is best captured by the F1-score of 0.68. While the model is highly sensitive, an F1-score below 0.70 indicates that the precision and recall trade-off is unbalanced [34]. The model learned the characteristics of a well-formed pulse but did not establish the boundaries of acceptable morphological distortion. In a live survey, this tendency would generate many false candidates and return the classification burden to human reviewers. Training on stable data alone therefore does not guarantee robust performance.

6.2 Interpretation of Reverse Generalisation

Scenario 2 reversed the training configuration. The network was trained on the highly distorted, narrow profiles of PSR J1929+1357 and tested on the stable emissions of PSR J1910+0517. Recall fell to 17 per cent, with the model missing 70 genuine pulses. The network had evidently learned the sharp, peaked spikes of the variable pulsar as its reference and rejected the broader, lower-gradient profiles of the stable source.

This behaviour produced perfect precision (100 per cent), generating no false positives. However, this performance degradation is reflected in the minority class F1-score of just 0.29. This low mean underscores that perfect precision is functionally useless if the model's sensitivity is compromised, as the core purpose of a blind sky survey is to discover faint or unexpected sources. The architecture memorised particular scintillation patterns rather than the core neutron star emission morphology, confirming that data diversity strongly dictates network generalisation.

6.3 The Efficacy of the Unified Approach

Scenario 3 addressed the biases observed in the isolated tests by merging both datasets. Exposure to the full range of morphological variation encouraged the network to identify features common to both emission regimes rather than relying on source-specific characteristics [35].

The model achieved 83 per cent recall and 84 per cent precision on the combined test set, yielding an overall accuracy of 97 per cent. Most importantly, the minority class F1-score reached 0.83. In highly imbalanced astronomical datasets, the minority F1-score is a far more robust indicator of true model performance than overall accuracy, which is heavily skewed by the majority noise class. An F1-score of 0.83 confirms an equilibrium between sensitivity and specificity [36]. The unified approach therefore offers a practical route for turning deep learning from an experimental tool into a functional filter for large-scale radio surveys.

6.4 Comparison with Existing Literature

Standard matched-filter pipelines, which dominate traditional pulsar searches, rely heavily on absolute signal-to-noise ratio thresholds [37]. These methods often discard faint intermittent pulses that fall below fixed cut-offs.

Recent studies have moved toward feature-based approaches to overcome this limitation. While work by Lin and Li [31] demonstrated that deep learning can handle highly imbalanced pulsar candidate data, their methodology still heavily relied on engineered feature selection prior to classification. The present research interrogates and bypasses this reliance by proving that raw, normalised geometric topology alone is sufficient for a convolutional network to extract its own hierarchical features.

While Li et al. [38] argued that pulse morphology provides a metric for variable sources, their models were primarily tested on stable, high-signal profiles. This study challenges the assumption that models trained on stable profiles can inherently generalise to noisy environments, as demonstrated by the catastrophic failure observed in Scenario 1. Therefore, while the literature broadly agrees on the utility of neural networks, this study highlights a critical gap in current research paradigms. Models must be explicitly trained on high-variance, scintillating data to be viable for real-world blind sky surveys.

6.5 Cross-Disciplinary Parallels in Time-Series Analysis

Intermittent pulsar detection shares notable similarities with seismological event monitoring. Seismologists commonly use the Short-Term Average to Long-Term Average (STA/LTA) algorithm to identify transient events such as microseismicity or volcanic explosions within noisy continuous recordings [39]. The algorithm computes the ratio between a short moving window (capturing instantaneous amplitude) and a long-term background window, an event is flagged when this ratio exceeds a threshold.

While computationally efficient and suitable for real-time use, STA/LTA shares a similar core weakness as traditional pulsar matched filters, heavy dependence on absolute amplitude and signal-to-noise ratios. It requires extensive manual parameter tuning and still misses weak events in high-noise conditions. The one-dimensional convolutional neural network developed here avoids these issues. Instead of relying on rolling averages that sudden noise bursts can corrupt, the network examines the entire phase window at once. This spatial perspective helps distinguish genuine emission spikes from baseline shifts. Combined with min-max normalisation and a focus on geometric topology, the approach provides a robust method for classifying events in chaotic time-series data.

6.6 Methodological Limitations

A primary analytical limitation of this study is the absence of formal statistical significance testing. While the F1-scores and confusion matrices provide a clear descriptive overview of model performance across the three scenarios, non-parametric tests, such as McNemar’s test for paired nominal data or bootstrapping confidence intervals for the minority F1-scores, were not applied. Consequently, while the unified approach appears vastly superior to the isolated models, the statistical confidence of this performance gap remains unquantified. Future iterations must incorporate rigorous significance testing to definitively validate architectural improvements against baseline matched-filter pipelines.

The pipeline achieved strong classification metrics, yet the data processing methodology has several inherent constraints. The most significant arises from the frequency integration step described in Chapter 2. Collapsing multiple frequency channels into a single broadband time profile permanently erases frequency-dependent scintillation structures. Terrestrial radio frequency interference is typically narrow-band, whereas genuine astrophysical emission spans the entire observing band [23]. Reducing the data to one dimension helped avoid overfitting, but it removed a key diagnostic for separating real signals from Earth-based noise. Any interference that mimics a pulse shape in the integrated profile could therefore evade detection. Future work could address this by adopting a two-dimensional convolutional architecture that retains frequency-time structure, provided sufficiently large training datasets are available to prevent overfitting.

A further limitation concerns the use of inverse-frequency class weighting to handle dataset imbalance. While this approach stabilised training in the present experiments, its behaviour at scale is less certain. Applying heavy numerical penalties to large, unlabelled survey archives containing diverse and unscrubbed noise could substantially increase false-positive rates. The method assumes that the training data adequately represent the full range of noise morphologies likely to be encountered, an assumption that cannot be guaranteed in practice. Deployment on an observatory with a different noise floor might cause the pre-calculated weights to produce over- or under-prediction of the minority class. Addressing this would require either continuous recalculation of weights or the adoption of adaptive loss functions that respond to the live signal-to-noise characteristics of each observation.

7 Conclusions and Future Work

7.1 Conclusions

The primary aim of this research was to address the classification difficulties posed by intermittent pulsars. Traditional detection pipelines rely heavily on absolute signal-to-noise ratio thresholds and frequently fail to isolate faint, variable pulses from background noise. To overcome this limitation, the project developed an automated classification framework based on supervised deep learning. The pipeline removed absolute flux density variations through min-max normalisation, enabling a one-dimensional Convolutional Neural Network to classify signals solely according to their geometric topology.

Isolated testing scenarios highlighted important aspects of convolutional learning. Training exclusively on stable emissions produced a model that was overly permissive and generated many false positives. Training on highly variable emissions, by contrast, resulted in a rigid model that missed most genuine pulses. The unified training approach mitigated these generalisation issues. By presenting the network with the full range of morphological variation from both sources, it learned invariant pulse features instead of source-specific distortions. On the combined holdout dataset the model achieved 97 per cent overall accuracy.

This work demonstrates that morphology-based deep learning offers a practical alternative to traditional matched filtering. The network balanced sensitivity and precision effectively, indicating that such frameworks can evolve from experimental tools into reliable components for large-scale radio surveys.

7.2 Future Work

Several methodological constraints identified in this study warrant further attention. The most notable limitation arises from the frequency integration step. Collapsing multiple frequency channels into a single broadband profile removes the frequency-time structure that helps distinguish genuine broadband pulsar emission from narrow-band terrestrial interference.

Future versions of the pipeline should therefore explore a two-dimensional convolutional architecture that processes uncollapsed frequency-phase data. This would enable the network to identify radio frequency interference more effectively. Such an upgrade would require substantially larger labelled datasets of intermittent pulsars to avoid overfitting in the deeper network.

Another area for improvement concerns the use of inverse-frequency class weighting. While effective for the current experiments, fixed weights may become unstable when applied to large, unlabelled survey archives with varying noise characteristics. Subsequent studies could investigate adaptive loss functions, such as focal loss, that adjust dynamically to the signal-to-noise properties of each observation without relying on pre-computed static weights.

Finally, a hybrid deployment strategy could combine the strengths of classical and deep learning methods. The Short-Term Average to Long-Term Average (STA/LTA) algorithm is computationally lightweight and effective at detecting amplitude anomalies in continuous time-series. A practical pipeline could use STA/LTA as a fast initial trigger to flag candidate windows across wide-field surveys. The Convolutional Neural Network would then examine only these selected windows, applying detailed morphological classification to reduce false alarms. This combined approach would improve both efficiency and reliability for future large-scale pulsar searches.

References

- [1] Baade, W., Zwicky, F., 1934, Remarks on Supernovae and Cosmic Rays, *Physical Review*, **46**, 76 to 77.
- [2] Hewish, A., Bell, S. J., Pilkington, J. D. H., Scott, P. F., Collins, R. A., 1968, Observation of a Rapidly Pulsating Radio Source, *Nature*, **217**, 709 to 713.
- [3] Maund, J. R., Crowther, P. A., Janka, H. T., Langer, N., 2017, Bridging the gap: from massive stars to supernovae, *Philosophical Transactions of the Royal Society A*, **375**, 20170025.
- [4] de Oliveira, H. O., Magalhaes, N. S., 2021, Angular momentum conservation and core superfluid dynamics for the pulsar J1734 minus 3333, *Astronomische Nachrichten*, **342**, 255 to 258.
- [5] Losovsky, B., 2022, The specifics of pulsar radio emission, *Open Astronomy*, **31**, 189 to 197.
- [6] Vohl, D., van Leeuwen, J., Maan, Y., 2024, Topology of Pulsar Profiles (ToPP) I. Graph theory method and classification of the EPN, *Astronomy and Astrophysics*, **687**, A113.
- [7] van den Heuvel, E. P. J., Taam, R. E., 1984, Two types of binary radio pulsars with different evolutionary histories, *Nature*, **309**, 235 to 237.
- [8] Damour, T., 2015, 1974: the discovery of the first binary pulsar, *Classical and Quantum Gravity*, **32**, 124009.
- [9] Good, D. C., Andersen, B. C., Chawla, P., Crowter, K., Dong, F. Q., Fonseca, E., Meyers, B. W., Ng, C., Pleunis, Z., Ransom, S. M., 2021, First Discovery of New Pulsars and RRATs with CHIME/FRB, *The Astrophysical Journal*, **922**, 43.
- [10] Oppenheimer, J. R., Volkoff, G. M., 1939, On Massive Neutron Cores, *Physical Review*, **55**, 374 to 381.
- [11] Bors, D., Stańczy, R., 2024, Tolman Oppenheimer Volkoff Equation, *arXiv preprint*, arXiv:2408.09751.
- [12] Gold, T., 1968, Rotating Neutron Stars as the Origin of the Pulsating Radio Sources, *Nature*, **218**, 731 to 732.
- [13] Venturi Piñas, F., Yip, A. K. L., Cheong, P. C. K., Ruiz, M., 2026, Impact of Rotation on Magnetic Field Stability and Orientation in Isolated Neutron Stars, *The Astrophysical Journal*, **1001**, 49.
- [14] Lorimer, D. R., Kramer, M., 2004, Handbook of Pulsar Astronomy, *Cambridge University Press*, Cambridge.
- [15] Lyne, A. G., Stappers, B. W., Freire, P. C. C., Hessels, J. W. T., Kaspi, V. M., Allen, B., Zhu, W. W., 2017, Two long term intermittent pulsars discovered in the PALFA Survey, *The Astrophysical Journal*, **834**, 72.
- [16] Gendre, M. A., Fenech, D. M., Beswick, R. J., Muxlow, T. W. B., Argo, M. K., 2013, Flux density variations of radio sources in M82 over the last three decades, *Monthly Notices of the Royal Astronomical Society*, **431**, 1107 to 1120.

- [17] Morello, V., Barr, E. D., Bailes, M., Flynn, C. M., Keane, E. F., van Straten, W., 2014, SPINN: a straightforward machine learning solution to the pulsar candidate selection problem, *Monthly Notices of the Royal Astronomical Society*, **443**, 1651 to 1662.
- [18] Lee, K. J., Guillemot, L., Yue, Y. L., Kramer, M., Champion, D. J., 2012, Application of the Gaussian mixture model in pulsar astronomy pulsar classification and candidates ranking for the Fermi 2FGL catalogue, *Monthly Notices of the Royal Astronomical Society*, **424**, 2832 to 2840.
- [19] Gao, S. J., Li, X. D., Yan, Z., Shao, Y. X., Zhou, P., 2025, RRAT J2325 minus 0530: A Rotating Radio Transient with an Atypical Waiting time Distribution, *The Astrophysical Journal*, **991**, 201.
- [20] Hotan, A. W., van Straten, W., Manchester, R. N., 2004, PSRCHIVE and PSRFITS: An Open Approach to Radio Pulsar Data Storage and Analysis, *Publications of the Astronomical Society of Australia*, **21**, 302 to 309.
- [21] Ransom, S. M., Eikenberry, S. S., Middleditch, J., 2002, Fourier techniques for very long astrophysical time series analysis, *The Astronomical Journal*, **124**, 1788.
- [22] Wang, Y. C., Li, M. T., Pan, Z. C., Zheng, J. H., 2019, Pulsar candidate classification with deep convolutional neural networks, *Research in Astronomy and Astrophysics*, **19**, 133.
- [23] Eatough, R. P., Keane, E. F., Lyne, A. G., 2009, An interference removal technique for radio pulsar searches, *Monthly Notices of the Royal Astronomical Society*, **395**, 410 to 415.
- [24] Hofmann, T., Schölkopf, B., Smola, A. J., 2008, Kernel methods in machine learning, *Ann. Statist.*, **36**, 1171 to 1220.
- [25] Mizutani, H., 1994, The back propagation method for CNN, *Proceedings of IEEE International Symposium on Circuits and Systems ISCAS '94*, 463 to 466.
- [26] Yan, M., Yang, J., Chen, C., Zhou, J. T., Pan, Y., Zeng, Z., 2021, Enhanced gradient learning for deep neural networks, *IET Image Processing*, **16**, 365 to 377.
- [27] Wang, Z., Li, H., 2020, Research on a convolution operation method based on domain transformation in deep learning, *Journal of Physics Conference Series*, **1550**, 032132.
- [28] Ide, H., Kurita, T., 2017, Improvement of learning for CNN with ReLU activation by sparse regularization, *2017 International Joint Conference on Neural Networks (IJCNN)*, 2684 to 2691.
- [29] Lee, E., Rustam, F., Aljedaani, W., Ishaq, A., Rupapara, V., Ashraf, I., 2021, Predicting Pulsars from Imbalanced Dataset with Hybrid Resampling Approach, *Advances in Astronomy*, **2021**, 4916494.
- [30] Ozturk, O., Kavlak, K. B., 2019, Statistical Inference Using Stratified Ranked Set Samples From Finite Populations, *Ranked Set Sampling*.
- [31] Lin, H., Li, X., 2024, Dealing with the data imbalance problem on pulsar candidates sifting based on feature selection, *Research in Astronomy and Astrophysics*.
- [32] Kingma, D. P., Ba, J., 2015, Adam: A Method for Stochastic Optimization, *3rd International Conference for Learning Representations*.

- [33] Sujon, K. M., Hassan, R., Choi, K., et al., 2025, Accuracy, precision, recall, f1 score, or MCC? empirical evidence from advanced statistics, ML, and XAI for evaluating business predictive models, *J Big Data*, **12**, 268.
- [34] Hand, D. J., Christen, P., Kirielle, N., 2021, F*: an interpretable transformation of the F measure, *Machine Learning*, **110**, 451 to 456.
- [35] Najafzadeh, H., Raissi, Z., Golmohammady, S., Safari Kaji, P., Esmaeili, M., 2025, Deep learning for classifying quantum emission signals in WS2 monolayers using wavelet transform, *Scientific Reports*, **15**, 41470.
- [36] Sánchez Reolid, R., López de la Rosa, F., López, M. T., Fernández Caballero, A., 2022, One dimensional convolutional neural networks for low/high arousal classification from electrodermal activity, *Biomedical Signal Processing and Control*, **71**, 103203.
- [37] Huang, Y. J., Hanna, C., Ewing, B., Godwin, P., Gonsalves, J., Magee, R., Messick, C., Tsukada, L., Yarbrough, Z., Joshi, P., Kennington, J., Niu, W., Rollins, J., Shah, U., 2025, Scalable matched filtering pipeline for gravitational wave searches of compact binary mergers, *Physical Review D*, **112**, 082002.
- [38] Li, G., Watanabe, K., Anzai, H., Song, X., Qiao, A., Ohta, M., 2019, Pulse Wave Pattern Classification with a Convolutional Neural Network, *Scientific Reports*, **9**, 14930.
- [39] Fang, Z., Cheng, H., Wang, X., Luo, C., 2025, Research on STA/LTA Microseismic Arrival Time Picking Method Based on Variational Mode Decomposition, *Applied Sciences*, **15**, 13220.

A Project Source Code

Due to the extensive length of the data processing and plotting scripts, the complete executable Python codebase, including Jupyter Notebooks and environment dependencies, is hosted permanently on GitHub.

The full repository can be accessed at: https://github.com/DanielBarranco-Phy/automated_classification_of_intermittent_pulsars_utilising_convolutional_neural_networks

To demonstrate proof of work, the core script utilised for the model architecture and training dynamics is provided below.

A.1 Model Architecture and Training Script

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import tensorflow as tf
4 from tensorflow.keras.models import Sequential
5 from tensorflow.keras.layers import Conv1D, MaxPooling1D, Flatten, Dense, Dropout,
    Input
6 from sklearn.model_selection import train_test_split
7 from sklearn.utils import class_weight
8
9 # =====
10 # 1. LOAD DATA
11 # =====
12 print("LOADING DATA...")
13 # Load the newly created MASTER datasets
14 X = np.load('J1910_Master_data_1D.npy')
15 y = np.load('J1910_Master_labels_1D.npy')
16
17 # Reshape for CNN: (Samples, 512) -> (Samples, 512, 1)
18 # CNNs expect a 3D input: Batch x Steps x Channels
19 X = X.reshape((X.shape[0], X.shape[1], 1))
20
21 print(f"Data Loaded. Shape: {X.shape}")
22
23 # =====
24 # 2. SPLIT TRAINING / VALIDATION
25 # =====
26 # Keep 20 percent of J1910 aside to check progress during training
27 X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state
    =42, stratify=y)
28
29 print(f"Training samples: {len(X_train)} ({np.sum(y_train==1)} Pulses)")
30 print(f"Validation samples: {len(X_val)} ({np.sum(y_val==1)} Pulses)")
31
32 # =====
33 # 3. COMPUTE CLASS WEIGHTS
34 # =====
35 # Since there are fewer Pulses (1) than Noise (0), we weight Pulses higher.
36 weights = class_weight.compute_class_weight(
37     class_weight='balanced',
38     classes=np.unique(y_train),
```

```

39     y=y_train
40 )
41 class_weights = dict(enumerate(weights))
42 print(f"\nClass Weights: {class_weights}")
43
44 # =====
45 # 4. 1D CNN MODEL
46 # =====
47 model = Sequential([
48     # Input Layer
49     Input(shape=(512, 1)),
50
51     # Feature Extraction
52     Conv1D(filters=16, kernel_size=16, activation='relu'),
53     MaxPooling1D(pool_size=4),
54
55     Conv1D(filters=32, kernel_size=8, activation='relu'),
56     MaxPooling1D(pool_size=4),
57
58     # Classification
59     Flatten(),
60     Dense(64, activation='relu'),
61     Dropout(0.5), # Prevents memorization
62     Dense(1, activation='sigmoid') # Binary Output
63 ])
64
65 model.compile(optimizer='adam',
66               loss='binary_crossentropy',
67               metrics=['accuracy', tf.keras.metrics.Recall(name='recall')])
68
69 print("\n== MODEL SUMMARY ==")
70 model.summary()
71
72 # =====
73 # 5. TRAIN THE MODEL
74 # =====
75 print("\n STARTING TRAINING...")
76 history = model.fit(
77     X_train, y_train,
78     epochs=50,
79     batch_size=16,
80     validation_data=(X_val, y_val),
81     class_weight=class_weights,
82     verbose=1
83 )
84
85 # =====
86 # 6. PLOT TRAINING HISTORY
87 # =====
88 plt.figure(figsize=(12, 4))
89
90 # Plot Loss
91 plt.subplot(1, 2, 1)
92 plt.plot(history.history['loss'], label='Train Loss', color='blue')

```

```

93 plt.plot(history.history['val_loss'], label='Val Loss', color='orange')
94 plt.title('Model Loss (Lower is Better)')
95 plt.xlabel('Epoch')
96 plt.ylabel('Loss')
97 plt.legend()
98 plt.grid(alpha=0.3)
99
100 # Plot Recall
101 plt.subplot(1, 2, 2)
102 plt.plot(history.history['recall'], label='Train Recall', color='green')
103 plt.plot(history.history['val_recall'], label='Val Recall', color='red')
104 plt.title('Recall / Sensitivity (Higher is Better)')
105 plt.xlabel('Epoch')
106 plt.ylabel('Recall')
107 plt.legend()
108 plt.grid(alpha=0.3)
109
110 plt.tight_layout()
111 plt.show()

```